HAQMS Final Assignment Documentation

This document summarizes the engineering work completed for the Figital Labs Full Stack Web Development Internship Assignment.

Objective

The assignment required improving an existing hospital management system that intentionally contained security vulnerabilities, frontend issues, performance bottlenecks, database inefficiencies, concurrency problems, and incomplete features.

The goal was not to fix every issue, but to demonstrate debugging ability, prioritization, code understanding, and practical full-stack improvements.

Issues Identified

Frontend Issues

1. Some pages had poor text contrast, making dashboard, physician cards, reports, and queue monitor sections hard to read.
2. The dashboard had a React hooks-order error caused by returning before all hooks were executed.
3. The patient medical history route was incomplete and caused a 404-style page.
4. The landing page looked too empty and did not provide a strong first impression.
5. The public queue monitor did not clearly explain or display live queue state.
6. Some UI sections were not responsive enough for smaller screens.

Backend Issues

1. The public queue endpoint originally required authentication, which prevented the public monitor from loading correctly.
2. Queue mutations did not send real-time updates to the frontend.
3. Some API responses are inconsistent across routes.
4. Some routes still contain intentionally vulnerable or inefficient code from the original assignment, such as weak validation and legacy authorization behavior.

Database And Prisma Issues

1. The project required PostgreSQL setup through Prisma.
2. Prisma schema uses DATABASE_URL, so local environment configuration must be correct.
3. Some schema comments identify missing indexes and constraints.
4. Queue token generation has a known race-condition risk because it calculates the next token number through a read-then-create flow.

Git And Submission Issues

1. Generated folders like .next and node_modules were accidentally tracked.
2. GitHub rejected the push because some generated files were larger than 100 MB.
3. .env needed to stay local and not be committed.

Fixes Implemented

Frontend Improvements

1. Rebuilt the landing page with a polished hospital operations design.
2. Added rotating healthcare imagery and stronger call-to-action sections.
3. Improved dashboard readability with a dedicated dashboard-readable style layer.
4. Restyled physician registry cards with a professional light clinical palette.
5. Restyled admin audit report KPI cards and table for better readability.
6. Restyled the live queue monitor so headings, doctor names, idle state, and token pills are visible.
7. Added a patient history route at:

```
/patients/[id]/history-records
```

1. Fixed the dashboard React hooks-order error by ensuring all hooks run before any conditional return.
2. Improved responsive layout behavior for dashboard and landing sections.

Backend Improvements

1. Added Socket.IO server support on top of the Express server.
2. Added queue event emission for queue creation and updates.
3. Made the public queue read endpoint accessible without login.
4. Kept protected routes for queue mutation actions such as check-in and status updates.

Live Queue Improvements

1. Added Socket.IO client support on the frontend queue page.
2. The queue page now listens for:

```
queue:created
queue:updated
queue:changed
```

1. Added fallback polling so the queue still refreshes if WebSocket connection fails.
2. Queue tokens are grouped by doctor.
3. CALLING tokens appear in the "Now Calling" area.

4. WAITING tokens appear in the "Queue List" area.

Database Setup

1. Configured PostgreSQL through Prisma using:

   ```
   DATABASE_URL
   ```

1. Added backend/.env.example for safe environment setup.
2. Added backend/prisma/init.sql as a manual SQL setup fallback.
3. Preserved seeded demo users, doctors, patients, appointments, and queue tokens.

GitHub Submission Fixes

1. Added .gitignore.
2. Removed .next, node_modules, and .env from Git tracking.
3. Kept generated files local only.
4. Successfully pushed the cleaned project to the alok branch.

Optimizations Performed

1. Improved frontend rendering stability by fixing the hooks-order bug.
2. Added WebSocket updates to reduce dependency on manual refresh.
3. Added fallback polling for reliability.
4. Improved user experience through clearer navigation, higher contrast, and better page structure.
5. Added documentation and diagrams so reviewers can understand the system quickly.

Remaining Known Issues

These are intentionally documented because the assignment states that fixing everything is not required.

1. Frontend still has some hardcoded local API fallbacks for development.
2. Some backend routes still return inconsistent response shapes.
3. Auth tokens are stored in localStorage; production systems should prefer a more secure cookie-based approach.
4. Some backend routes contain intentionally weak validation from the original assignment.
5. Queue token generation can still create duplicate token numbers under concurrent check-ins.
6. Prisma schema comments identify missing indexes and missing uniqueness constraints.
7. Full production deployment configuration still needs environment variables for hosted frontend and backend URLs.

Major Engineering Decisions

Prioritized User-Visible Stability

The first priority was making the app runnable and demonstrable. Broken pages, unreadable UI, missing routes, and queue access issues were fixed before deeper backend refactors.

Preserved Assignment Intent

The original project intentionally contained vulnerable and inefficient areas. Instead of hiding every issue, the project now documents remaining risks clearly while improving enough features to demonstrate strong full-stack understanding.

Added Real-Time Queue Updates

Socket.IO was added because the hospital queue monitor is naturally a real-time feature. This demonstrates backend events, frontend subscriptions, and state synchronization.

Kept Demo Accounts Visible

Seeded login accounts are visible in the UI so evaluators can quickly test Admin, Doctor, and Receptionist workflows without database inspection.

How To Verify

Backend

```
cd backend
npm install
npx prisma generate
npx prisma migrate dev
node prisma/seed.js
npm run dev
```

Backend:

```
http://localhost:5000
```

Frontend

```
cd frontend
npm install
npm run dev
```

Frontend:

```
http://localhost:3000
```

Database

Check data in Prisma Studio:

```
cd backend
npx prisma studio
```

Important tables:

```
User
Doctor
Patient
Appointment
QueueToken
```

Live Queue

1. Open:

```
http://localhost:3000/queue
```

2. Login to the dashboard in another tab.
3. Check in a patient or update a token status.
4. The public queue page should update through Socket.IO.

Submission Checklist

1. GitHub repository updated.
2. Documentation prepared.
3. Video walkthrough should be recorded.
4. Deployed application URL still needs to be added after deployment.
5. Documentation link can point to this file or the GitHub README.